

Full Source Code – CrewAI Study Buddy

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

Complete Project with OpenAI

agents.yaml

Found in folder: **/src/buddy_ai/config**

This code defines five roles for a language model: PDF Content Extractor, AI Content Summarizer, Quiz Creator, Quiz Master, and Flashcard Generator. Each role has a specific goal, backstory, and configuration settings, such as the language model to use and verbosity level. These roles are used to automate tasks in a study aid tool.

pdf_reader:

role: >

PDF Content Extractor

goal: >

Extract the entire text content from a fixed PDF document and provide it.

backstory: >

You are skilled at retrieving text from PDF documents.

The PDF is located at C:\Users\user\Downloads\Documents\sample_document.pdf.

llm: gpt-4o

verbose: **false**

function_calling_llm: **null**

summarizer:

role: >

AI Content Summarizer

goal: >

Summarize the given text into a concise and informative overview.

backstory: >

You specialize **in** taking long pieces **of** text and producing a clear, concise summary.

Your summaries highlight the key points **while** maintaining factual accuracy.

llm: gpt-4o

verbose: **false**

function_calling_llm: **null**

quiz_maker:

role: >

Quiz Creator

goal: >

Create a quiz (e.g., multiple-choice questions) based on given summarized text.

Do not show the answers **of** the questions to the user.

backstory: >

Expert at formulating questions that test understanding **of** text content.

llm: gpt-4o

verbose: **false**

function_calling_llm: **null**

study_buddy:

role: >

Quiz Master

goal: >

Interactively test the user based on the quiz generated.

backstory: >

You are a helpful study buddy that asks questions to the user and checks their answer.

llm: gpt-4o

verbose: **false**

function_calling_llm: **null**

flashcard_maker:

role: >

Flashcard Generator

goal: >

Create flashcards from the summarized text to help the user study.

backstory: >

You turn summarized text into helpful question-answer pairs, each representing a flashcard.

llm: gpt-4o

verbose: **false**

strict_parser: **false**

function_calling_llm: **null**

tasks.yaml

Found in folder: **/src/buddy_ai/config**

This code defines a series of tasks to process a PDF document, including extracting text, summarizing content, generating quizzes and flashcards, and interacting with the user to test their understanding. Each task specifies an agent responsible for executing the task, the expected output, and any required context or human input. The tasks are defined in a YAML file, which provides a structured format for describing the workflow.

reading_task:

description: >

Extract the entire text from the PDF document at
C:\Users\user\Downloads\Documents\sample_document.pdf.

Return the raw text content without modification.

expected_output: >

The complete extracted text **of** the PDF document.

agent: pdf_reader

summarizing_task:

description: >

Take the provided PDF text and summarize it into a concise overview that highlights the main points.

expected_output: >

A concise summary (around 2-4 paragraphs) highlighting the key themes and facts from the document.

agent: summarizer

context:

- reading_task

output_file: summary.md

quiz_task:

description: >

Using the summary **of** the PDF's content, create a quiz with multiple-choice questions that test understanding of the main concepts.

Provide say, 5 multiple-choice questions. Each question should have 4 options and indicate the correct answers.

expected_output: >

A quiz with 5 multiple-choice questions and their correct answers.

agent: quiz_maker

context:

- summarizing_task

output_file: quiz.md

user_test_task:

description: >

Present the quiz questions to the user and ask them to provide their answers.

After the user responds, check the correctness and provide a final score.

expected_output: >

A summary of user performance, e.g. "You answered X out of Y questions correctly."

agent: study_buddy

context:

- quiz_task

human_input: true

flashcard_task:

description: >

Using the summary of the PDF's content, create 5 flashcards. Each card has a "front" (question or term) and a "back" (explanation).

expected_output: >

A structured list or text specifying each flashcard's front and back.

agent: flashcard_maker

context:

- summarizing_task

output_file: flashcards.md

study_flashcards_task:

description: >

Go through the flashcards one by one with the user. The user can type "show" to display a card's front,

"flip" to reveal its back, or "quit" to exit.

expected_output: >

Interactive session with the user. No final text is required, just user engagement.

agent: study_buddy

context:

- flashcard_task

human_input: **true**

crew.py

Found in folder: **/src/buddy_ai**

This code defines a class `BuddyAi` that represents a crew in the CrewAI framework. The class defines several agents and tasks that work together to process a PDF document, including reading the document, summarizing its content, generating a quiz, and creating flashcards. The tasks are executed in a sequential process, and the crew is configured using YAML files.

```
import os
```

```
from crewai import Agent, Crew, Process, Task
```

```
from crewai.project import CrewBase, agent, crew, task
```

```
from crewai_tools import PDFSearchTool
```

```
# If you want to run a snippet of code before or after the crew starts,
```

```
# you can use the @before_kickoff and @after_kickoff decorators
```

```
# https://docs.crewai.com/concepts/crews#example-crew-class-with-decorators
```

```
PDF_PATH = 'C:\\Users\\user\\Downloads\\Documents\\sample_document.pdf'
```

```
@CrewBase
```

```
class BuddyAi():
```

```
    """BuddyAi crew"""
```

```
    # Learn more about YAML configuration files here:
```

```
    # Agents: https://docs.crewai.com/concepts/agents#yaml-configuration-recommended
```

```
    # Tasks: https://docs.crewai.com/concepts/tasks#yaml-configuration-recommended
```

```
    agents_config = 'config/agents.yaml'
```

```
    tasks_config = 'config/tasks.yaml'
```

```
    # If you would like to add tools to your agents, you can learn more about it here:
```

```
    # https://docs.crewai.com/concepts/agents#agent-tools
```

@agent

def pdf_reader(self) -> Agent:

```
return Agent(  
    config=self.agents_config['pdf_reader'],  
    verbose=False,  
    tools=[PDFSearchTool(pdf=PDF_PATH)]  
)
```

@agent

def summarizer(self) -> Agent:

```
return Agent(  
    config=self.agents_config['summarizer'],  
    verbose=False  
)
```

@agent

def quiz_maker(self) -> Agent:

```
return Agent(  
    config=self.agents_config['quiz_maker'],  
    verbose=False,  
    strict_parser=False  
)
```

@agent

def study_buddy(self) -> Agent:

```
return Agent(  
    config=self.agents_config['study_buddy'],  
    verbose=False,  
    tools=[PDFSearchTool(pdf=PDF_PATH)]  
)
```

```

    config=self.agents_config['study_buddy'],
    verbose=False,
)
@agent
def flashcard_maker(self) -> Agent:
    return Agent(
        config=self.agents_config['flashcard_maker'],
        verbose=False,
        strict_parser=False,
        function_calling_llm=None
    )
# To learn more about structured task outputs,
# task dependencies, and task callbacks, check out the documentation:
# https://docs.crewai.com/concepts/tasks#overview-of-a-task
@task
def reading_task(self) -> Task:
    """
    This task will:
    - verify file existence and access
    - attempt multiple queries to extract full content
    - Return extracted content or clear out all error messages
    """
    def reading_logic(agent, inputs):
        # 0. verify the file path and acces
        if not os.path.isfile(PDF_PATH):
            return "ERROR: PDF file not found or not accessible at the given path."

```



```

# 1. Attempt to retrieve content from the PDF using different queries.

queries = ["", "all content", "document", "*"]

extracted_content = None

fallback_snippet = "PDF RAG the PDFSearchTool is designed to search PDF files."

for q in queries:

    agent.think(f"Trying query '{q}' to get PDF content. ")

    result = agent.use_tool("Search a PDF's content", {"query": q})

    if result and fallback_snippet not in result:

        # trying to get something different from the fallback description

        extracted_content = result.strip()

        break


# 2. confirm that content was extracted

if not extracted_content:

    return (

        "ERROR: unable to extract meaningful text from the PDF."

        "Make sure that PDF has searchable text or try another document."

    )

return extracted_content

return Task(

    config=self.tasks_config['reading_task'],

    logic=reading_logic

)

@task

```

```

def summarizing_task(self) -> Task:
    """
    Summarize the extracted PDF text.

    If the reading_task returned an ERROR,
    handle that gracefully here!
    """

    def summarizing_logic(agent, inputs, context):
        pdf_text = context.get('reading_task', '')
        if pdf_text.startswith("ERROR:"):
            return f"Cannot summarize because: {pdf_text}"

        return agent.llm(
            f"Summarize the following text into a concise overview:\n\n{pdf_text}"
        ).strip

    return Task(
        config=self.tasks_config['summarizing_task'],
        logic=summarizing_logic,
        output_file='output/summary.md'
    )

@task
def quiz_task(self) -> Task:
    return Task(
        config=self.tasks_config['quiz_task'],
    )

@task

```

```

def user_test_task(self) -> Task:

    def test_logic(agent, inputs, context):

        quiz_text = context.get('quiz_task', '')

        # the agent will prompt the user:

        agent.think("I will now ask the user to answer the quiz.")

        agent.say("Here are your quiz questions:\n")

        agent.say(quiz_text)

        agent.say("Please provide your answers in the format: A,B,C,... for each question in
order.")

        user_answers = inputs.get('human_input')

        user_answers_list = [ans.strip().upper() for ans in user_answers.split(",")]

        # Q1, A, b, c, Correct answer: A

        import re

        correct_answers = re.findall(r'Correct Answer:\s*([A-D])', quiz_text, re.IGNORECASE)

        score = 0

        for user_ans, correct_ans in zip(user_answers_list, correct_answers):

            if user_ans == correct_ans.upper():

                score += 1

        total = len(correct_ans)

        result = f"You answered {score} out of {total} correctly!"

        agent.say(result)

        return result

    return Task(

        config=self.tasks_config['user_test_task'],

        logic=test_logic

```

```
)
```

```
@task
```

```
def flashcard_task(self) -> Task:
```

```
    return Task(
```

```
        config=self.tasks_config['flashcard_task']
```

```
)
```

```
@task
```

```
def study_flashcards_task(self) -> Task:
```

```
    def study_logic(agent, inputs, context):
```

```
        # parse the flashcards from the flashcard_task output
```

```
        # let the user type show, flip or quit to navigate them.
```

```
        flashcards_text = context.get('flashcards_task', '')
```

```
        if not flashcards_text.strip():
```

```
            return "No flashcards found. Possibly an error."
```

```
        agent.think("Parsing flashcards from text...")
```

```
        # 1) front: ...
```

```
        #  back: ...
```

```
        lines = [l.strip() for l in flashcards_text.split('\n') if l.strip()]
```

```
        flashcards = []
```

```
        current_fc = {}
```

```
        for line in lines:
```

```
            if line.lower().startswith("front:"): 
```

```
                current_fc = {"front": line[6:].strip(), "back": ""}
```

```
            elif line.lower().startswith("back:"): 
```

```
                current_fc["back"] = line[5:].strip()
```

```
flashcards.append(current_fc)

current_fc = {}

if not flashcards:

    return "Couldn't parse any flashcards from the text."


agent.say(f"I have {len(flashcards)} flashcards loaded. We'll go through them now.\n")
agent.say("Type 'show' to see the next flashcard, 'flip' to see its back, or 'quit' to end.\n")
index = 0

showing_front = False

user_input = inputs.get('human_input', "").lower()

if user_input == "quit":

    return "Exiting flashcard study session."


if index >= len(flashcards):

    return "No more flashcards left."


if user_input == "show":

    agent.say(f"Flashcard {index + 1} front: {flashcards[index]['front']}")

    showing_front = True

    return "Type 'flip' next time to see the back, or 'quit' to stop."

elif user_input == "flip" and not showing_front:

    return "You need to 'show' the card first. Then you can 'flip' it."

elif user_input == "flip" and showing_front:

    # show the back

    agent.say(f"Flashcard {index+1} back: {flashcards[index]['back']}")

    return f"Now you can type 'show' to move to the next card or 'quit' to stop."
```

```

    else:

        return "Invalid command. Please type 'show' or 'flip' or 'quit'"

    return Task(

        config=self.tasks_config['study_flashcards_task'],

        logic=study_logic,

        human_input=True

    )

@crew
def crew(self) -> Crew:

    """Creates the BuddyAi crew"""

    # To learn how to add knowledge sources to your crew, check out the documentation:

    # https://docs.crewai.com/concepts/knowledge#what-is-knowledge

    return Crew(

        agents=self.agents, # Automatically created by the @agent decorator

        tasks=self.tasks, # Automatically created by the @task decorator

        process=Process.sequential,

        verbose=True,

        # process=Process.hierarchical, # In case you wanna use that instead

        https://docs.crewai.com/how-to/Hierarchical/

    )

```

main.py

Found in folder: **/src/buddy_ai**

This code defines four functions: run, train, replay, and test, which interact with the BuddyAi crew to perform different actions. The functions take varying inputs, such as the number of iterations or a task ID, and execute the corresponding crew action. Each function also includes error handling to catch and raise exceptions if any issues occur during execution.

```

#!/usr/bin/env python

import sys

import warnings

from buddy_ai.crew import BuddyAi

warnings.filterwarnings("ignore", category=SyntaxWarning, module="pysbd")

def run():
    """
    Run the crew.
    """
    # No inputs needed since tasks are self-contained
    inputs = {}

    BuddyAi().crew().kickoff(inputs=inputs)

def train():
    """
    Train the crew for a given number of iterations.
    """
    inputs = {}

    try:
        BuddyAi().crew().train(n_iterations=int(sys.argv[1]), filename=sys.argv[2],
inputs=inputs)

    except Exception as e:
        raise Exception(f"An error occurred while training the crew: {e}")

def replay():
    """
    Replay the crew execution from a specific task.
    """

```

```

try:

    BuddyAi().crew().replay(task_id=sys.argv[1])

except Exception as e:

    raise Exception(f"An error occurred while replaying the crew: {e}")

def test():

    """

    Test the crew execution and returns the results.

    """

    inputs = {}

    try:

        BuddyAi().crew().test(n_iterations=int(sys.argv[1]), openai_model_name=sys.argv[2],
inputs=inputs)

    except Exception as e:

        raise Exception(f"An error occurred while replaying the crew: {e}")

```

Complete Project with Local LLM

agents.yaml

Found in folder: **/src/buddy_ai/config**

This code defines five roles for a project, each with a specific goal and backstory. The roles include a PDF content extractor, AI content summarizer, quiz creator, quiz master, and flashcard generator. Each role is configured to use the Llama 3.2 model and has various settings such as verbosity and function calling.

pdf_reader:

role: >

PDF Content Extractor

goal: >

Extract the entire text content from a fixed PDF document and provide it.

Use Llama 3.2 model.

backstory: >

You are skilled at retrieving text from PDF documents.

The PDF is located at C:\Users\user\Downloads\Documents\sample_document.pdf.

llm: llama-3.2

verbose: **false**

function_calling_llm: **null**

summarizer:

role: >

AI Content Summarizer

goal: >

Summarize the given text into a concise and informative overview.

backstory: >

You specialize **in** taking long pieces **of** text and producing a clear, concise summary.

Your summaries highlight the key points **while** maintaining factual accuracy.

llm: llama-3.2

verbose: **false**

function_calling_llm: **null**

quiz_maker:

role: >

Quiz Creator

goal: >

Create a quiz (e.g., multiple-choice questions) based on given summarized text.

Do not show the answers **of** the questions to the user.

backstory: >

Expert at formulating questions that test understanding **of** text content.

llm: llama-3.2

verbose: **false**

function_calling_llm: **null**

study_buddy:

role: >

Quiz Master

goal: >

Interactively test the user based on the quiz generated.

backstory: >

You are a helpful study buddy that asks questions to the user and checks their answer.

llm: llama-3.2

verbose: **false**

function_calling_llm: **null**

flashcard_maker:

role: >

Flashcard Generator

goal: >

Create flashcards from the summarized text to help the user study.

backstory: >

You turn summarized text into helpful question-answer pairs, each representing a flashcard.

llm: llama-3.2

verbose: **false**

strict_parser: **false**

function_calling_llm: **null**

tasks.yaml

Found in folder: **/src/buddy_ai/config**

This code defines a series of tasks to process a PDF document, including extracting text, summarizing content, generating a quiz, creating flashcards, and interacting with the user to test their knowledge. Each task is described with its expected output and the agent responsible for executing it. The tasks are linked together through context dependencies, allowing them to build on each other's outputs.

reading_task:

description: >

Extract the entire text from the PDF document at
C:\Users\user\Downloads\Documents\sample_document.pdf.

Return the raw text content without modification.

expected_output: >

The complete extracted text **of** the PDF document.

agent: pdf_reader

summarizing_task:

description: >

Take the provided PDF text and summarize it into a concise overview that highlights the main points.

expected_output: >

A concise summary (around 2-4 paragraphs) highlighting the key themes and facts from the document.

agent: summarizer

context:

- reading_task

output_file: summary.md

quiz_task:

description: >

Using the summary **of** the PDF's content, create a quiz with multiple-choice questions that test understanding of the main concepts.

Provide say, 5 multiple-choice questions. Each question should have 4 options and indicate the correct answers.

expected_output: >

A quiz with 5 multiple-choice questions and their correct answers.

agent: quiz_maker

context:

- summarizing_task

output_file: quiz.md

user_test_task:

description: >

Present the quiz questions to the user and ask them to provide their answers.

After the user responds, check the correctness and provide a final score.

expected_output: >

A summary of user performance, e.g. "You answered X out of Y questions correctly."

agent: study_buddy

context:

- quiz_task

human_input: true

flashcard_task:

description: >

Using the summary of the PDF's content, create 5 flashcards. Each card has a "front" (question or term) and a "back" (explanation).

expected_output: >

A structured list or text specifying each flashcard's front and back.

agent: flashcard_maker

context:

- summarizing_task

output_file: flashcards.md

study_flashcards_task:

description: >

Go through the flashcards one by one with the user. The user can type "show" to display a card's front,

"flip" to reveal its back, or "quit" to exit.

expected_output: >

Interactive session with the user. No final text is required, just user engagement.

agent: study_buddy

context:

- flashcard_task

human_input: **true**

crew.py

Found in folder: **/src/buddy_ai**

This code defines a class `BuddyAi` that represents a crew in the CrewAI framework. The class defines several agents and tasks that work together to process a PDF document, including reading the document, summarizing its content, generating quiz questions, and creating flashcards. The tasks are executed in a sequential process, and the crew is configured to use specific language models and tools.

```
import os
```

```
from crewai import Agent, Crew, Process, Task, LLM
```

```
from crewai.project import CrewBase, agent, crew, task
```

```
from crewai_tools import PDFSearchTool
```

```
# If you want to run a snippet of code before or after the crew starts,
```

```
# you can use the @before_kickoff and @after_kickoff decorators
```

```
# https://docs.crewai.com/concepts/crews#example-crew-class-with-decorators
```

```
PDF_PATH = 'C:\\Users\\user\\Downloads\\Documents\\sample_document.pdf'
```

```
@CrewBase
```

```
class BuddyAi():
```

```
    """BuddyAi crew"""
```

```
    # Learn more about YAML configuration files here:
```

```
    # Agents: https://docs.crewai.com/concepts/agents#yaml-configuration-recommended
```

```
    # Tasks: https://docs.crewai.com/concepts/tasks#yaml-configuration-recommended
```

```
    agents_config = 'config/agents.yaml'
```

```
    tasks_config = 'config/tasks.yaml'
```

```
    # If you would like to add tools to your agents, you can learn more about it here:
```

```
    # https://docs.crewai.com/concepts/agents#agent-tools
```

```
@agent
```

```
def pdf_reader(self) -> Agent:
```

```
    return Agent(
```

```
        config=self.agents_config['pdf_reader'],
```

```
        llm=LLM(
```

```
            model="ollama/llama3.2",
```

```
            base_url="http://localhost:11434"
```

```
        ),
```

```
        verbose=False,
```

```
        tools=[PDFSearchTool(pdf=PDF_PATH)]
```

```
    )
```

```
@agent
```

```
def summarizer(self) -> Agent:
```

```
    return Agent(
```

```
config=self.agents_config['summarizer'],  
llm=LLM(  
    model="ollama/llama3.2",  
    base_url="http://localhost:11434"  
),  
verbose=False  
)
```

@agent

def quiz_maker(self) -> Agent:

```
return Agent(  
    config=self.agents_config['quiz_maker'],  
    llm=LLM(  
        model="ollama/llama3.2",  
        base_url="http://localhost:11434"  
    ),  
    verbose=False,  
    strict_parser=False  
)
```

@agent

def study_buddy(self) -> Agent:

```
return Agent(  
    config=self.agents_config['study_buddy'],  
    llm=LLM(  
        model="ollama/llama3.2",
```

```

        base_url="http://localhost:11434"
    ),
    verbose=False,
)
@agent
def flashcard_maker(self) -> Agent:
    return Agent(
        config=self.agents_config['flashcard_maker'],
        llm=LLM(
            model="ollama/llama3.2",
            base_url="http://localhost:11434"
        ),
        verbose=False,
        strict_parser=False,
        function_calling_llm=None
    )
# To learn more about structured task outputs,
# task dependencies, and task callbacks, check out the documentation:
# https://docs.crewai.com/concepts/tasks#overview-of-a-task
@task
def reading_task(self) -> Task:
    """
    This task will:
    - verify file existence and access
    - attempt multiple queries to extract full content
    - Return extracted content or clear out all error messages
    """

```



```
"""
```

```
def reading_logic(agent, inputs):

    # 0. verify the file path and acces

    if not os.path.isfile(PDF_PATH):

        return "ERROR: PDF file not found or not accessible at the given path."


    # 1. Attempt to retrieve content from the PDF using different queries.

    # queries = ["", "all content", "document", "*"]

    extracted_content = None

    fallback_snippet = "PDF RAG the PDFSearchTool is designed to search PDF files."

    # for q in queries:

    agent.think(f"Trying query (all content) to get PDF content. ")

    result = agent.use_tool("Search a PDF's content", {"query": "all content"})

    if result and fallback_snippet not in result:

        # trying to get something different from the fallback description

        extracted_content = result.strip()


    # 2. confirm that content was extracted

    if not extracted_content:

        return (

            "ERROR: unable to extract meaningful text from the PDF."

            "Make sure that PDF has searchable text or try another document."

        )

    return extracted_content

return Task(

    config=self.tasks_config['reading_task'],
```

```
    logic=reading_logic
)
```

```
@task
```

```
def summarizing_task(self) -> Task:
```

```
    """
```

```
    Summarize the extracted PDF text.
```

```
    If the reading_task returned an ERROR,
    handle that gracefully here!
```

```
    """
```

```
def summarizing_logic(agent, inputs, context):
```

```
    pdf_text = context.get('reading_task', '')
```

```
    if pdf_text.startswith("ERROR:"):
```

```
        return f"Cannot summarize because: {pdf_text}"
```

```
    return agent.llm(
```

```
        f"Summarize the following text into a concise overview:\n\n{pdf_text}"
```

```
    ).strip
```

```
return Task(
```

```
    config=self.tasks_config['summarizing_task'],
```

```
    logic=summarizing_logic,
```

```
    output_file='output/summary.md'
```

```
)
```

```
@task
```

```
def quiz_task(self) -> Task:
```

```

return Task(
    config=self.tasks_config['quiz_task'],
)
@task
def user_test_task(self) -> Task:
    def test_logic(agent, inputs, context):
        quiz_text = context.get('quiz_task', '')
        # the agent will prompt the user:
        agent.think("I will now ask the user to answer the quiz.")
        agent.say("Here are your quiz questions:\n")
        agent.say(quiz_text)
        agent.say("Please provide your answers in the format: A,B,C,... for each question in
order.")
        user_answers = inputs.get('human_input')
        user_answers_list = [ans.strip().upper() for ans in user_answers.split(",")]
        # Q1, A, b, c, Correct answer: A
        import re
        correct_answers = re.findall(r'Correct Answer:\s*([A-D])', quiz_text, re.IGNORECASE)
        score = 0
        for user_ans, correct_ans in zip(user_answers_list, correct_answers):
            if user_ans == correct_ans.upper():
                score += 1
        total = len(correct_ans)
        result = f"You answered {score} out of {total} correctly!"
        agent.say(result)
    return result

```

```

return Task(
    config=self.tasks_config['user_test_task'],
    logic=test_logic
)
@task
def flashcard_task(self) -> Task:
    return Task(
        config=self.tasks_config['flashcard_task']
    )
@task
def study_flashcards_task(self) -> Task:
    def study_logic(agent, inputs, context):
        # parse the flashcards from the flashcard_task output
        # let the user type show, flip or quit to navigate them.
        flashcards_text = context.get('flashcards_task', '')
        if not flashcards_text.strip():
            return "No flashcards found. Possibly an error."

        agent.think("Parsing flashcards from text...")
        # 1) front: ...
        # back: ...

        lines = [l.strip() for l in flashcards_text.split('\n') if l.strip()]
        flashcards = []
        current_fc = {}
        for line in lines:

```

```

if line.lower().startswith("front:"):
    current_fc = {"front": line[6:].strip(), "back": ""}
elif line.lower().startswith("back:"):
    current_fc["back"] = line[5:].strip()
    flashcards.append(current_fc)
    current_fc = {}
if not flashcards:
    return "Couldn't parse any flashcards from the text."

agent.say(f"I have {len(flashcards)} flashcards loaded. We'll go through them now.\n")
agent.say("Type 'show' to see the next flashcard, 'flip' to see its back, or 'quit' to end.\n")
index = 0

showing_front = False

user_input = inputs.get('human_input', "").lower()
if user_input == "quit":
    return "Exiting flashcard study session."

if index >= len(flashcards):
    return "No more flashcards left."

if user_input == "show":
    agent.say(f"Flashcard {index + 1} front: {flashcards[index]['front']}")
    showing_front = True
    return "Type 'flip' next time to see the back, or 'quit' to stop."
elif user_input == "flip" and not showing_front:
    return "You need to 'show' the card first. Then you can 'flip' it."

```

```

elif user_input == "flip" and showing_front:

    # show the back

    agent.say(f"Flashcard {index+1} back: {flashcards[index]['back']}")

    return f"Now you can type 'show' to move to the next card or 'quit' to stop."

else:

    return "Invalid command. Please type 'show' or 'flip' or 'quit'"

return Task(

    config=self.tasks_config['study_flashcards_task'],

    logic=study_logic,

    human_input=True

)

@crew

def crew(self) -> Crew:

    """Creates the BuddyAi crew"""

    # To learn how to add knowledge sources to your crew, check out the documentation:

    # https://docs.crewai.com/concepts/knowledge#what-is-knowledge

    return Crew(

        agents=self.agents, # Automatically created by the @agent decorator

        tasks=self.tasks, # Automatically created by the @task decorator

        process=Process.sequential,

        verbose=True,

        # process=Process.hierarchical, # In case you wanna use that instead

        https://docs.crewai.com/how-to/Hierarchical/

    )

```

main.py

Found in folder: **/src/buddy_ai**

This code defines four functions: run, train, replay, and test, which interact with the BuddyAi crew to perform different actions. The functions take varying inputs, such as the number of iterations or a task ID, and execute the corresponding crew action. Each function also includes error handling to catch and raise exceptions if any issues occur during execution.

```
#!/usr/bin/env python

import sys

import warnings

from buddy_ai.crew import BuddyAi

warnings.filterwarnings("ignore", category=SyntaxWarning, module="pysbd")

def run():
    """
    Run the crew.
    """
    # No inputs needed since tasks are self-contained
    inputs = {}
    BuddyAi().crew().kickoff(inputs=inputs)

def train():
    """
    Train the crew for a given number of iterations.
    """
    inputs = {}
    try:
        BuddyAi().crew().train(n_iterations=int(sys.argv[1]), filename=sys.argv[2],
inputs=inputs)
    except Exception as e:
        raise Exception(f"An error occurred while training the crew: {e}")
```

```
def replay():
```

```
    """
```

```
    Replay the crew execution from a specific task.
```

```
    """
```

```
    try:
```

```
        BuddyAi().crew().replay(task_id=sys.argv[1])
```

```
    except Exception as e:
```

```
        raise Exception(f"An error occurred while replaying the crew: {e}")
```

```
def test():
```

```
    """
```

```
    Test the crew execution and returns the results.
```

```
    """
```

```
    inputs = {}
```

```
    try:
```

```
        BuddyAi().crew().test(n_iterations=int(sys.argv[1]), openai_model_name=sys.argv[2],  
inputs=inputs)
```

```
    except Exception as e:
```

```
        raise Exception(f"An error occurred while replaying the crew: {e}")
```